

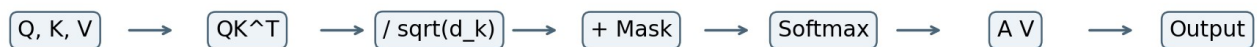
Scaled Dot-Product Attention

$QK^T \rightarrow \text{Scale} \rightarrow \text{Mask} \rightarrow \text{Softmax} \rightarrow AV$

深度分析 • Gotchas • Know-how • Know-why • 训练与推理系统

版本：2026-08 | 面向 Transformer / LLM 算法与 Infra 学习

Scaled Dot-Product Attention dataflow



核心视角

Attention 不是一张“相关性热力图”这么简单。它同时是：① learned content-addressable routing；② 概率归一化算子；③ 长序列的 $O(L^2)$ 计算核心；④ GPU 上典型的 IO-bound / memory-traffic 问题；⑤ 训练稳定性和长上下文行为的重要来源。

目录

1. 一页总览：Scaled Dot-Product Attention 到底在做什么
2. 数学定义、Tensor Shape 与复杂度
3. 为什么使用 Dot Product，而不是任意相似度
4. 为什么必须除以 $\sqrt{d_k}$
5. Mask：因果性、Padding、Sliding Window 与任意 Bias
6. Softmax：概率化、平移不变性与凸组合
7. Stable Softmax / LogSumExp：数值稳定的第一原则
8. Online Softmax：为什么能分块精确计算
9. 从 Attention Matrix 到输出 AV：信息路由的几何意义
10. 梯度、熵、饱和、温度与 Q/K 范数
11. 低精度：FP16/BF16/FP8 下哪些地方最危险
12. Naive Attention 的 GPU 成本模型
13. FlashAttention 1：IO-aware exact attention
14. FlashAttention 2/3：并行划分、Hopper 与低精度
15. PyTorch SDPA：math / memory-efficient / flash 后端与语义坑
16. Prefill、Decode、KV Cache 与 Attention kernel
17. Attention Sink：现象、机制与流式推理
18. 长上下文：稀释、位置、窗口与“有效上下文”
19. 从零实现：正确性优先的 PyTorch MVP
20. 生产级 Know-how：训练、推理、监控
21. Gotchas：高频错误清单
22. Know-why：关键因果关系
23. Ablation / Benchmark 设计
24. Debug Playbook：从 NaN 到 kernel mismatch
25. 面试与推导题
26. 参考文献与结论

1. 一页总览：Scaled Dot-Product Attention 到底在做什么

Transformer 的注意力核心由 Vaswani 等人在 2017 年给出：先计算 Query 与 Key 的点积相似度，除以 $\sqrt{d_k}$ ，再对 Key 维度做 softmax，最后用得到的权重对 Value 做加权求和。[1]

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}((\mathbf{QK}^T + \mathbf{B} + \mathbf{M}) / \sqrt{d_k}) \mathbf{V}$$

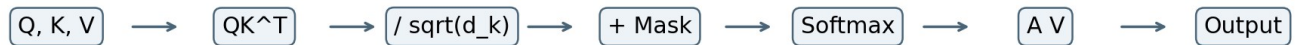
其中 B 可以表示相对位置 bias、ALiBi 等可加性偏置，M 可以表示 causal/padding/window 等 mask；不同实现可能把 scale 放在 Q、K 或 logits 上，但数学目标一致。

阶段	数学对象	核心问题	典型 Gotcha
$\mathbf{QK}^T$	raw logits S	“当前 query 应该从哪些 key 读取？”	shape/head 维度错、RoPE 后位置错
Scale	$S / \sqrt{d_k}$	控制 logit 方差与 softmax sharpness	误用 d_{model} 而非 d_{head}
Mask/Bias	$S + M + B$	约束可见性/注入先验	bool/additive mask 语义相反
Softmax	$A = \text{softmax}(S)$	将竞争关系归一化	全 mask 行、溢出、低精度 underflow
AV	$O = AV$	按权重读取 Value	dropout 时权重和不再为 1

Know-why

Q/K 决定“从哪里读”，V 决定“读回什么”；softmax 把每一行限制在概率单纯形上，所以每个 head 的输出（无 dropout 时）是 Value 向量的凸组合。这一结构约束既带来稳定、可解释的路由，也会产生“无法自然选择什么都不读”等副作用。

Scaled Dot-Product Attention dataflow



2. 数学定义、Tensor Shape 与复杂度

2.1 标准 MHA shape

对 batch B、query 长度 L_q 、key/value 长度 L_k 、head 数 H、每头维度 D_h ：

张量	典型 Shape	说明
Q	$[B, H, L_q, D_h]$	每个 query head 的查询向量
K	$[B, H, L_k, D_h]$	MHA 中 $Hk_v = H$ ；GQA/MQA 可更小
V	$[B, H, L_k, D_v]$	通常 $D_v = D_h$
$S = \mathbf{QK}^T$	$[B, H, L_q, L_k]$	attention logits
A	$[B, H, L_q, L_k]$	softmax 后权重
O	$[B, H, L_q, D_v]$	head 输出

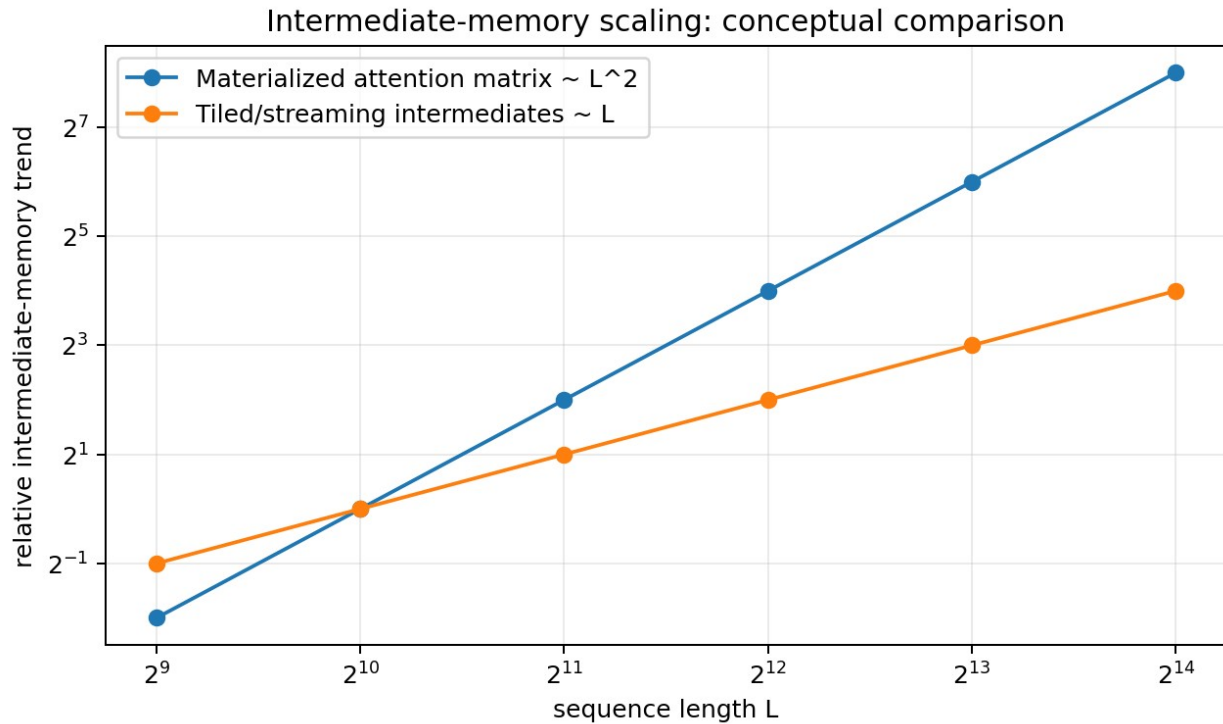
GQA 中多个 Q heads 共享同一组 K/V heads。概念实现常显式 repeat K/V，但生产 kernel 通常避免真实复制，以免浪费内存带宽。

2.2 FLOPs 与中间内存

$$QK^T \text{ FLOPs} \approx 2 \cdot B \cdot H \cdot L_q \cdot L_k \cdot D_h$$

$$AV \text{ FLOPs} \approx 2 \cdot B \cdot H \cdot L_q \cdot L_k \cdot D_v$$

全序列自注意力 $L_q=L_k=L$ 时，主计算复杂度 $O(L^2D)$ 。朴素实现若显式物化 S 和 A，则中间激活也随 $O(BHL^2)$ 增长。FlashAttention 的贡献不是把精确 attention 的算术复杂度改成线性，而是避免把完整 $L \times L$ 中间矩阵反复写入/读取 HBM，从而让中间内存接近线性并显著降低 IO。[3]



3. 为什么使用 Dot Product，而不是任意相似度

点积注意力可以看作可学习双线性匹配的最终阶段： $Q=XW_Q$ 、 $K=XW_K$ 后， $q_i^T k_j = x_i^T W_Q W_K^T x_j$ 。也就是说，模型先学习“在哪个表示空间比较”，再用高吞吐 GEMM 批量完成所有 pairwise 匹配。

- 计算友好： QK^T 可以直接映射到矩阵乘法，是 GPU/TPU 最成熟的高吞吐 primitive。
- 表示灵活： W_Q 、 W_K 已经把原始 hidden state 映射到任务所需匹配空间，点积本身无需复杂。
- 与 cosine 不同：标准 dot product 同时编码方向和范数；因此 Q/K norm 会影响 logit magnitude。
- 与 additive attention 相比：大维度批处理时 GEMM 更容易高效实现；原始 Transformer 明确指出 dot-product attention 在实践中更快、更节省空间。[1]

Gotcha

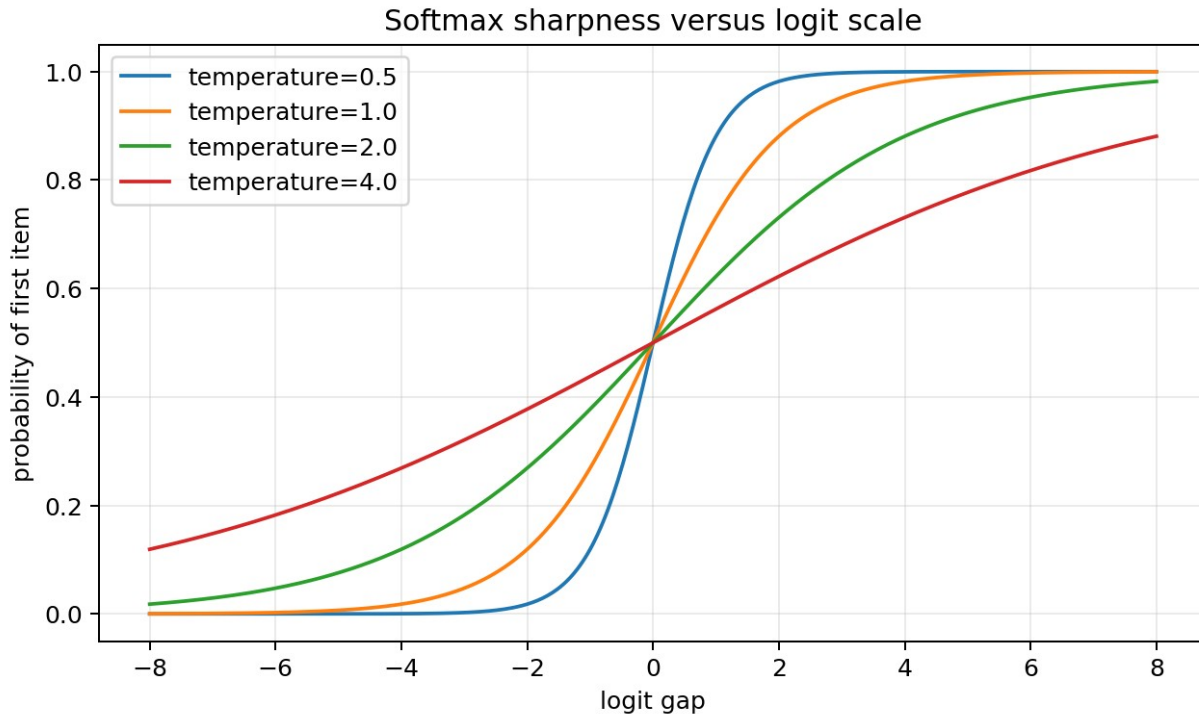
“Attention 权重高 = 两个 token 语义相似”并不成立。Q/K 是上下文、按 head 投影后的匹配向量；一个 head 可以学习指代、位置、格式、控制流或其他路由规则，而不是词向量相似度。

4. 为什么必须除以 $\sqrt{d_k}$

假设 q 与 k 的各维独立、均值 0、方差 1。单个点积：

$$s = q \cdot k = \sum_r q_r k_r$$

在近似独立条件下 $\text{Var}(q_r k_r) \approx 1$ ，于是 $\text{Var}(s) \approx d_k$ ，标准差约 $\sqrt{d_k}$ 。 d_k 越大，未经缩放的 logits 越容易出现较大绝对值；softmax 会进入接近 one-hot 的饱和区，非最大项梯度变小。原始 Transformer 因此使用 $1/\sqrt{d_k}$ 缩放。 [1]



错误/变体	后果
除以 $\sqrt{d_{\text{model}}}$	若 $d_{\text{model}} \gg d_{\text{head}}$ ，会过度平滑 attention
完全不 scale	head_dim 增大时 logits 易饱和
把 scale 重复应用两次	attention 过平，训练行为改变
Q/K 已做强归一化但仍照搬旧温度	可能需要重新校准 entropy/temperature

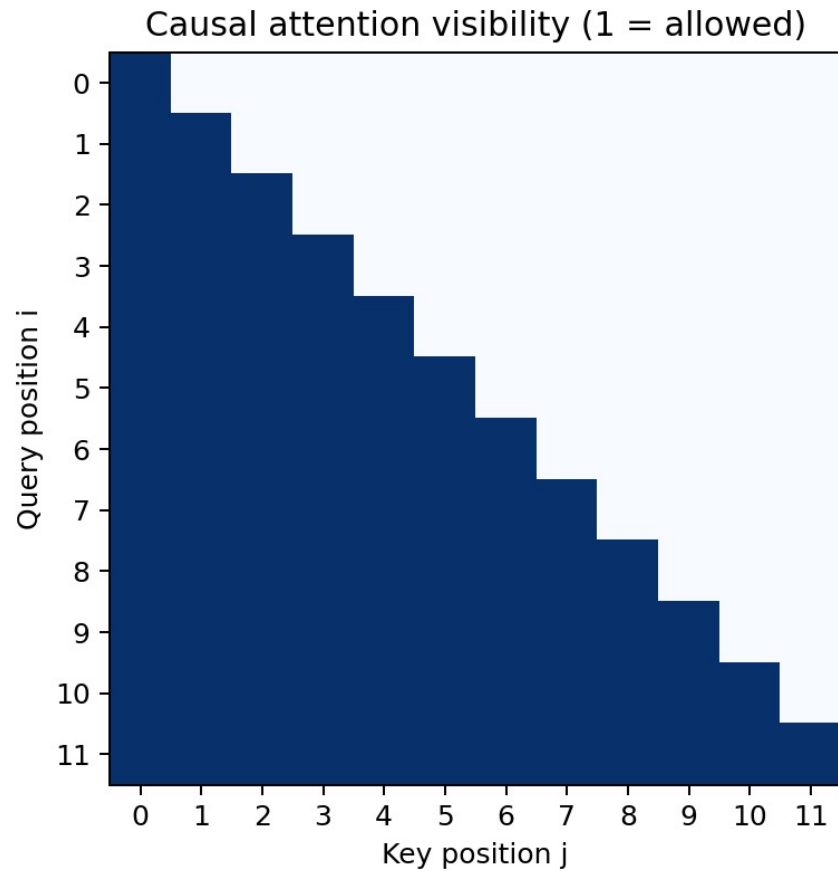
Know-how

不要只监控 attention probability。训练时同步记录 pre-softmax logits 的 mean/std/max、Q/K RMS、attention entropy；否则“softmax 看起来正常”可能掩盖 Q/K norm drift。

5. Mask：因果性、Padding、Sliding Window 与任意 Bias

5.1 Causal mask

Decoder-only LM 在位置 i 只能访问 $j \leq i$ 。数学上，未来位置的 logit 加上 $-\infty$ ，softmax 后精确变为 0。实际低精度 kernel 通常用数据类型可表示的极小值或专用 causal 路径，而不是在 Python 中真正创建 $-\infty$ 矩阵。



5.2 Mask 族谱

类型	可见性	典型用途	关键风险
Causal	$j \leq i$	自回归 LM	off-by-one; cache offset 错
Padding	屏蔽 PAD key	变长 batch	query padding 与 key padding 混淆
Sliding window	$ i-j $ 或 $i-j \leq W$	局部 attention	不同库边界定义差 1
Prefix/Block	按片段开放	prefix LM / packed sequences	跨样本泄漏
Arbitrary bias	连续值，不一定是 0/-∞	ALiBi/relative bias	与 scale 的先后/单位不一致

高频 API Gotcha

不同 API 的 bool mask 语义可能相反：有的 True 表示“允许注意”，有的 True 表示“禁止注意”。PyTorch `scaled_dot_product_attention` 的文档示例将 bool mask 的 False 位置填为 $-\infty$ ；迁移代码时必须做一个 3×3 手算单元测试，而不是凭记忆。[7]

6. Softmax：概率化、平移不变性与凸组合

$$a_j = \exp(s_j) / \sum_t \exp(s_t)$$

- 非负： $a_j \geq 0$ 。
- 归一化： $\sum_j a_j = 1$ 。
- 平移不变： $\text{softmax}(s+c) = \text{softmax}(s)$ 。

- 排序保持：更大 logit 对应更大概率。

于是单个 query/head 的输出 $o = \sum_j a_j v_j$ （在 attention dropout 之前）位于 Value 向量的凸包内。它不能通过权重产生负系数或直接把所有 Value 权重设为零。现代 attention sink 研究进一步指出，这种概率单纯形约束与某些“默认/无操作”行为之间存在结构性联系；2026 年理论工作在特定 trigger-conditional 任务上证明了 softmax 模型必须形成 sink，而非归一化 ReLU attention 可避免。[11] 该结论属于较新的理论研究，应与通用工程事实区分。

Know-why

Softmax 不只是“把分数变成概率”。它规定了 head 的路由几何：所有 key 竞争固定总质量 1。这让模型必须把注意力质量分配到某处，即使当前 query 没有任何值得读取的 key。

7. Stable Softmax / LogSumExp：数值稳定的第一原则

直接计算 $\exp(1000)$ 会溢出。利用平移不变性，令 $m = \max_j s_j$ ：

$$\text{softmax}(\mathbf{s}) = \exp(\mathbf{s} - \mathbf{m}) / \sum \exp(\mathbf{s} - \mathbf{m})$$

此时最大指数为 $\exp(0)=1$ ，其余项 ≤ 1 ，大幅降低 overflow 风险。对应的 log-normalizer：

$$\log \sum \exp(s_j) = m + \log \sum \exp(s_j - m)$$

这就是 logsumexp 技巧。它是安全 softmax、cross-entropy 以及 FlashAttention online normalization 的基础。

现象	原因	诊断
NaN	全 mask 行导致 0/0；exp overflow；上游 Q/K 已 NaN	检查每行是否至少一个有效 key；检查 pre-softmax finite
概率全 0/1	logit gap 太大或低精度 underflow	监控 entropy、max-min gap
不同 kernel 结果有微小差异	reduction 顺序与累加精度不同	用容差测试，不要要求 bitwise equal
训练突然发散	Q/K norm drift 导致 logits 爆大	Q/K RMS、logit std、grad norm 联合看

8. Online Softmax：为什么能分块精确计算

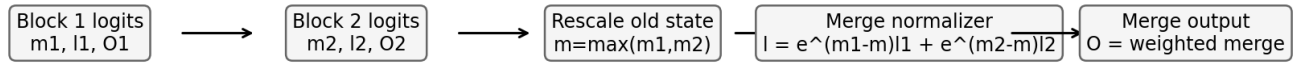
传统 safe softmax 至少需要先找 max，再求指数和；若 attention logits 太大而无法整体存入片上内存，就需要分块。Milakov 与 Gimelshein 2018 提出的 online normalizer 能在单次流式扫描中维护 running max 与 normalizer，减少内存访问。[2]

假设前一块状态为 $(m_{\text{old}}, l_{\text{old}})$ ，新块最大值 m_{blk} ，指数和 l_{blk} 。合并最大值 $m_{\text{new}} = \max(m_{\text{old}}, m_{\text{blk}})$ ，旧 normalizer 必须按新的坐标系重缩放：

$$l_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) \cdot l_{\text{old}} + \exp(m_{\text{blk}} - m_{\text{new}}) \cdot l_{\text{blk}}$$

对于 Attention，还要对 partial output 同样重缩放。只要对 running max、normalizer 和 partial output 正确 rescale，就可以改变遍历/分块顺序而保持数学等价（浮点舍入误差除外）。

Online softmax: retain only running max, normalizer, and partial output



Know-why

FlashAttention 能“边算 softmax 边算 AV”而无需保存完整 A，关键不是近似，而是 softmax normalizer 具有可合并的在线状态。

9. 从 Attention Matrix 到输出 AV：信息路由的几何意义

对第 i 个 query: $o_i = \sum_j a_{ij} v_j$ 。Q/K 负责选择路由权重，V 负责承载内容。即使两个模型有非常相似的 attention map，只要 V 投影不同，最终信息流也可能完全不同。

- Attention map 是中间路由系数，不等于最终贡献；贡献还受 V 的方向/范数与后续 W_O 影响。
- 单个 head 的 o 在 V 的凸包内，但多个 head concat 后经过 W_O ，可产生更一般的线性组合。
- Attention dropout 通常施加在 softmax 权重之后，训练时会破坏“每行和为 1”的瞬时性质；期望由 dropout scaling 保持。

Gotcha

不要用 attention probability 单独做“token 重要性”结论。它不是梯度归因，也不是因果贡献；至少要结合 V 、 W_O 、residual path，必要时用 activation patching/ablation 等更直接方法。

10. 梯度、熵、饱和、温度与 Q/K 范数

softmax Jacobian 为 $J = \text{diag}(a) - aa^T$ 。分布过尖时，大部分非最大项 $a \approx 0$ ，相关梯度很小；过平时，每个 key 的区分度不足。Attention 的“合适 sharpness”是优化和表达能力之间的平衡。

指标	过低/过高意味着什么	可能原因
Attention entropy	低=极尖；高=近均匀	QK norm、scale、位置 bias、训练阶段
max attention prob	长期 ≈ 1 可能饱和	Q/K norm drift、头塌缩
logit std	过大 $\rightarrow$ softmax 尖锐	scale 错、QKNorm 缺失/异常
Q RMS / K RMS	持续增长会放大 logits	初始化、norm、weight decay、优化器
head output RMS	过大/过小影响 residual	V/W_O 、dropout、head collapse

温度 T 可写成 $\text{softmax}(S/T)$ 。 $T < 1$ 更尖， $T > 1$ 更平。标准 $1/\sqrt{d_k}$ 等价于与 head dimension 相关的固定温度校准。QK-Norm 等现代变体则更直接控制 Q/K 范数，再配合可学习或固定 scale。

11. 低精度：FP16/BF16/FP8 下哪些地方最危险

Attention 同时包含 GEMM、max reduction、exp、sum reduction、除法/归一化和第二次 GEMM；不同子操作对数值精度的敏感性不同。

位置	风险	工程策略
QK^T accum	长维度累加误差	Tensor Core 常用低精度乘法 + 更高精度累加
pre-softmax logits	范围过大	scale、QKNorm、clamp 需谨慎
exp	overflow/underflow	subtract max；内部转 FP32 常见
normalizer sum	大量小项累加	FP32 accumulation/online rescale
$A \times V$	权重低精度量化误差	核实现选择、FP8 block scaling
mask sentinel	-1e9 在某些 dtype 行为不同	使用 kernel/API 规定的 mask 语义

FlashAttention-3 针对 Hopper GPU 通过异步流水、Tensor Core/TMA 协同和 FP8 block quantization 提速，并报告 FP16 最高约 740 TFLOPs/s、FP8 接近 1.2 PFLOPs/s，同时针对 FP8 设计降低数值误差。[5] 这些数字依赖特定硬件和 benchmark，不应直接外推到其他 GPU。

12. Naive Attention 的 GPU 成本模型

朴素实现常写成三个大步骤： $S=QK^T$ ； $A=\text{softmax}(S)$ ； $O=AV$ 。问题在于 S 、 A 都是 $B \times H \times L \times L$ ，随着 L 增长迅速成为 HBM 流量和激活内存瓶颈。

```

S = Q @ K.transpose(-2, -1)          # [B,H,L,L]
S = S * (1.0 / sqrt(d_head))
S = S + mask
A = softmax(S, dim=-1)                # [B,H,L,L]
O = A @ V                             # [B,H,L,Dh]
```

即使算术 FLOPs 已经很大，GPU 的现实性能仍受 memory hierarchy 影响。FlashAttention 论文的核心观点就是：分析 attention 时必须把 HBM↔SRAM IO 纳入复杂度模型，而不是只数 FLOPs。[3]

Know-how

性能分析至少同时报告：tokens/s、TFLOPs/s、HBM bytes、kernel time、max memory、sequence length、head_dim、causal/non-causal、dtype、dropout、GQA ratio。只说“FlashAttention 快 2x”没有可迁移意义。

13. FlashAttention 1：IO-aware exact attention

FlashAttention 将 Q 、 K 、 V 分块搬到片上 SRAM，在块内计算 QK^T 、在线 softmax 统计量和 partial O ，再把最终 O 写回 HBM。它不需要在 HBM 中物化完整 S/A 。论文将其描述为 IO-aware exact attention，并分析了在一定 SRAM 条件下的 IO 复杂度最优性。[3]

1. 按 Q block 与 K/V block 进行 tiling。
2. 块内计算局部 logits。
3. 用 running max / normalizer 合并 softmax 状态。
4. 同步重缩放 partial output。
5. 反向传播时可重新计算部分中间量，以计算换取显存/IO。

朴素 attention	FlashAttention
--------------	----------------

显式写 S 到 HBM	S 只在 tile 内短暂存在
显式写 A 到 HBM	A 不完整物化
多次遍历大矩阵	融合/分块减少 HBM traffic
中间内存 $O(L^2)$	中间内存近似 $O(L)$
数学结果	数学结果相同（浮点顺序不同）

14. FlashAttention 2/3：并行划分、Hopper 与低精度

14.1 FlashAttention-2

FlashAttention-2 继续保持精确 attention，但减少非 matmul FLOPs、改进 thread block 并行和 warp 内工作划分。论文报告相对 FlashAttention 约 $2\times$ kernel speedup，并在 A100 上达到理论峰值 FLOPs 的 50%-73% 区间。[4]

14.2 FlashAttention-3

FlashAttention-3 面向 Hopper：利用 Tensor Cores 与 TMA 的异步能力进行 warp specialization、重叠数据移动与计算，并针对 FP8 引入 block quantization / incoherent processing。[5]

版本	核心改进	不要误解
FA1	IO-aware tiling + online softmax	不是近似/稀疏 attention
FA2	更好的并行与 work partitioning	数学定义仍不变
FA3	Hopper async + FP8 低精度优化	特定硬件收益不能平移 to 所有 GPU

15. PyTorch SDPA：math / memory-efficient / flash 后端与语义坑

PyTorch 的 `torch.nn.functional.scaled_dot_product_attention` 提供统一语义，并可在满足条件时选择优化后端；官方教程和文档明确说明 `MultiheadAttention/TransformerEncoderLayer` 已使用该接口的优化实现。[7][8]

```
import torch.nn.functional as F

out = F.scaled_dot_product_attention(
    q, k, v,
    attn_mask=mask,
    dropout_p=p if training else 0.0,
    is_causal=True,
    enable_gqa=True,
)
```

Gotcha	为什么危险	测试建议
dropout_p 在 eval 仍显式传 >0	函数会按参数执行 dropout	eval 时手动传 0.0
is_causal 与 attn_mask 组合限制	不同版本/后端支持差异	锁定 PyTorch 版本并跑 backend matrix
bool mask 语义	与某些旧 API 相反	3×3 手算
GQA 支持条件	head 数整除与 backend 限制	MHA/GQA/MQA 单测
kernel 自动选择	机器/shape/dtype 变化可能换 backend	记录实际 backend 与版本

16. Prefill、Decode、KV Cache 与 Attention kernel

Prefill 与 decode 的形状完全不同。Prefill 通常 $L_q \approx L_k = L$ ，可形成大 GEMM；decode 每步 $L_q=1$ 、 L_k 随上下文增长，计算更像读整个 KV cache 后做一个窄矩阵乘法，因此往往更 memory-bandwidth bound。

阶段	L_q	L_k	主要瓶颈	常见优化
Training/prefill	大	大	$O(L^2)$ compute+IO	FlashAttention、sequence/context parallel
Decode	1 或小 batch token	很大	KV cache 读取带宽	GQA/MQA、paged KV、KV quantization
Chunked prefill	中等 block	历史+当前	调度与位置/causal 边界	chunk-aware mask/kernel
Speculative verify	多 token	历史+draft	批量验证效率	specialized attention scheduling

Critical parity test

同一模型、同一序列：一次性 full forward 的最后 k 个 hidden/logits，应与 prefill + token-by-token cached decode 在合理浮点容差内一致。若不一致，优先怀疑 position offset、causal mask、KV 写入索引、GQA head mapping，而不是“随机数”。

17. Attention Sink：现象、机制与流式推理

StreamingLLM 观察到：仅保留最近窗口的 KV 会在超过缓存长度后明显退化，但若额外保留最开始少量 token 的 KV，可大幅恢复性能；这些初始 token 常承接很高 attention，即使语义并不重要，因此称 attention sinks。[6]

- 工程含义：滑动窗口 KV eviction 不能简单“只保留最近 W 个 token”。
- 机制研究：后续工作发现 sink 更像 key-side bias/额外 attention score 存储点，且与 softmax 归一化结构有关。[10]
- 2026 理论结果：在特定任务族中，softmax 概率单纯形约束可迫使模型形成 sink；这支持“模型需要某处存放 no-op attention mass”的解释。[11]

Gotcha

不要把 attention sink 等同于“第一个 token 很重要”。它可能是路由/归一化的结构性垃圾桶；删除它的 Value 信息可能影响不大，但删除其 Key 可能破坏分布。

18. 长上下文：稀释、位置、窗口与“有效上下文”

把最大 context window 从 8K 改到 128K，不只改变 RoPE/位置范围，也改变每个 query 要竞争的 key 数。Softmax 的归一化分母随候选集合变化；若有效 logit margin 不随长度增长，概率质量可能被更多 key 稀释。2025 年“Softmax is not Enough”从 size generalisation 角度分析了这一类 sharpness 问题。[12]

问题	位置编码能否单独解决	还需要
位置超出训练范围	部分能（如 RoPE scaling）	长序列训练/微调
信息在 100K 处可寻址	不保证	data curriculum、retrieval-like training
softmax 候选变多导致稀释	不能完全	logit margin/temperature/architecture

$O(L^2)$ 成本	不能	Flash/sparse/window/hybrid
KV cache 线性增长	不能	GQA/MQA/quantization/offload

Know-why

“长上下文”至少是四个问题：地址是否可表示、相关 token 是否能被选中、softmax 是否保持足够 sharpness、系统是否付得起 compute/IO/KV 成本。把它们混成一个 max_position_embeddings 参数会造成大量误判。

19. 从零实现：正确性优先的 PyTorch MVP

```
import math
import torch

def sdpa_reference(q, k, v, mask=None, causal=False):
    # q: [B,H,Lq,D], k/v: [B,H,Lk,D]
    scale = 1.0 / math.sqrt(q.size(-1))
    logits = torch.matmul(q.float(), k.float().transpose(-2, -1)) * scale

    if causal:
        Lq, Lk = q.size(-2), k.size(-2)
        # For simple full self-attention only; cached decode needs offset-aware mask.
        cm = torch.ones((Lq, Lk), dtype=torch.bool, device=q.device).tril()
        logits = logits.masked_fill(~cm, float('-inf'))

    if mask is not None:
        if mask.dtype == torch.bool:
            logits = logits.masked_fill(~mask, float('-inf'))
        else:
            logits = logits + mask.float()

    probs = torch.softmax(logits, dim=-1)
    out = torch.matmul(probs, v.float())
    return out.to(v.dtype), probs
```

参考实现故意将 logits/softmax 提升到 float32，以优先得到可解释正确性基线；生产 kernel 会使用更复杂的 dtype/accumulation 策略。

19.1 最小测试集

- 无 mask：与 torch SDPA math backend 对比。
- causal 3×3：手算每一行允许位置。
- 全相同 logits：softmax 应均匀。
- 单个超大 logit：输出应接近对应 V。
- 加任意常数 c 到整行 logits：概率不变。
- GQA：显式 repeat K/V 的参考结果与 enable_gqa 对比。
- cached decode：full forward parity。
- 极端 logits / 全 mask 行：显式检查 finite/NaN 行为。

20. 生产级 Know-how：训练、推理、监控

20.1 训练前

- 锁定 head_dim、scale、mask 语义，并在 CPU/FP32 reference 上验证。
- 分别测试 causal、padding、packed sequence、variable length。

- 确定 attention dropout 的位置与 eval 行为。
- 记录使用的 kernel/backend 与版本，不要把 API 名当 kernel 名。
- 对 BF16/FP16 做 forward/backward tolerance test。

20.2 训练中

监控项	建议粒度	用途
Q/K/V RMS	layer/head sample	发现 norm drift
logit mean/std/max	layer/head sample	发现 scale/saturation
attention entropy	head sample	发现过尖/过平/塌缩
max prob / top-k mass	head sample	看路由集中度
output RMS	layer	residual stability
NaN/Inf counters	kernel/layer	第一时间定位数值错误
kernel time	shape bucket	性能回归

20.3 推理

- 按 prefill/decode 分开 benchmark。
- 记录 KV dtype、layout、page size、GQA ratio。
- 长上下文测试不能只测 throughput，还要测 retrieval/needle/multi-hop 等有效性。
- 切换 Flash/SDPA/xFormers/Triton kernel 前必须做 logits parity，而不只是 generation 看起来差不多。

21. Gotchas：高频错误清单

#	Gotcha	说明
1	Scale 用错维度	应基于 d_head/d_k，不是 d_model。
2	Scale 两次	上游已缩放 Q 或 kernel 内 scale，又手动除 $\sqrt{d}$ 。
3	Causal mask off-by-one	当前 token 是否允许看自己必须明确。
4	Cached decode mask 不考虑 offset	Lq=1 时简单 tril() 可能只允许第一个 key。
5	Bool mask 语义反了	True=allow/deny 在 API 间不统一。
6	全 mask 行	softmax($-\infty, \dots, -\infty$) 可能 NaN。
7	Padding mask 只处理 key 未处理 loss/query	PAD query 仍可能产生无意义输出。
8	Packed samples 串联泄漏	block diagonal mask 没构造对。
9	Sliding window 边界差 1	inclusive/exclusive 约定不同。
10	Additive bias 与 scale 顺序错	bias 的单位属于 logit space，缩放先后改变强度。
11	RoPE 与 mask position 不一致	位置坐标和缓存物理索引混淆。
12	GQA 显式 repeat K/V	功能对但性能/显存差。
13	Attention dropout eval 不关	某些函数按 dropout_p 参数而非 module.training。
14	Softmax dim 错	错误地跨 head/query 归一化。
15	FP16 exp 溢出	未 subtract max 或 kernel path 异常。
16	Mask sentinel 不适配 dtype	-1e9/最低有限值行为需确认。

17	直接比较不同 kernel bitwise	reduction 顺序不同应使用容差。
18	把 FA 当近似算法	FA 是 exact attention 的重排。
19	只看 FLOPs 判断性能	HBM traffic 常是关键。
20	把 memory O(L) 误解成 compute O(L)	FA 算术仍为 $O(L^2)$ 。
21	Decode 复用 prefill benchmark	两者瓶颈不同。
22	KV cache layout 与 kernel 假设不匹配	head-major/token-major/page layout 影响巨大。
23	Attention map=解释	忽略 V/W_O/residual。
24	Entropy 越低越好	过尖可能是饱和/塌缩。
25	Entropy 越高越好	过平可能无法检索。
26	第一个 token 高注意=语义重要	可能只是 sink。
27	窗口删除 sink	Streaming 模型性能可能突然恶化。
28	只扩 RoPE 就宣称长上下文	有效检索/softmax sharpness/训练分布未解决。
29	kernel fallback 未监控	某个 shape 可能从 Flash 悄悄退到 math backend。
30	Head dim 不受 kernel 支持	导致 fallback 或低效路径。
31	dropout/mask 导致 kernel eligibility 变化	性能测试必须覆盖真实配置。
32	非 contiguous stride	触发额外 copy 或不支持路径。
33	Autocast 预期错误	内部 accumulation dtype 与输入 dtype 不等。
34	checkpoint/recompute 与 FA 叠加误估收益	FA backward 已会重算部分量。
35	训练/推理实现不一致	训练用一种 mask/bias, serve 用另一种。
36	Batch variable length 只 padding	浪费 L^2 ; 可考虑 varlen kernel。
37	Cross-attention 误套 causal	encoder-decoder 可见性不同。
38	Loss 正常就不监控 attention	局部数值异常可能先积累后爆发。
39	把 top-1 prob 当路由质量	任务可能需要分布式多 token 聚合。
40	Softmax 前 clamp 过强	能“治 NaN”但可能改变模型容量。
41	任意换 sigmoid/ReLU attention	不是 kernel 优化, 而是架构/训练目标改变。
42	把理论 sink 必要性过度外推	2026 证明针对特定任务设置。
43	缺少 full-vs-cache parity	最常见 serving correctness 漏洞。
44	Benchmark 不报告 shape	attention 性能高度 shape-specific。
45	Benchmark 不 warmup/sync	GPU 异步导致错误时间。

22. Know-why：关键因果关系

问题	Know-why
为什么 QK^T	投影后用 GEMM 实现批量 content-addressable matching。
为什么 $\sqrt{d_k}$	控制点积方差随维度增长。
为什么 subtract max	softmax 平移不变, 可避免 exp overflow。
为什么 online softmax 可行	max 与 normalizer 可在新基准下重缩放并合并。
为什么 FlashAttention 快	减少 HBM↔SRAM IO 和中间矩阵物化, 不改变数学定义。

为什么 FA 内存近线性	不持久保存 $L \times L$ S/A。
为什么 decode 常 memory-bound	$L_q \approx 1$ ，算术强度低但需读大量历史 KV。
为什么 GQA 帮 decode	减少每 token 要读取的 K/V head 数。
为什么 entropy 能诊断	它直接反映 softmax 路由分布的尖锐程度。
为什么 attention sink 可能出现	softmax 必须把总质量 1 分配到某处；no-op 需要稳定“垃圾桶”。
为什么 mask 用 $-\infty$	使 $\exp(\text{masked})=0$ ，从归一化分母中彻底移除。
为什么全 mask 行危险	分母为 0，概率无定义。
为什么长上下文可能稀释	候选 key 数增加，但 logit margin 未必同步增大。
为什么位置 bias 进入 logits	它直接改变 query 对 key 的竞争分数。
为什么 kernel 结果不 bitwise 一致	浮点加法非结合，分块/reduction 顺序不同。
为什么 attention map 不等于贡献	最终贡献还乘 V，并经 $W_O/\text{residual}$ 。
为什么 softmax 输出是凸组合	权重非负且和为 1。
为什么低精度 softmax 常内部 FP32	$\exp/\text{reduction}$ 对动态范围和累加误差更敏感。
为什么不能只看 max context config	地址、训练分布、sharpness、系统成本是不同问题。
为什么 reference implementation 必须存在	高性能 kernel 太复杂，必须有可验证数学基线。

23. Ablation / Benchmark 设计

任何 attention 改动应把“模型质量”“数值稳定”“吞吐/显存”三个维度分开评估。

Ablation 轴	候选值	至少记录
Attention backend	math / memory-efficient / FA2 / FA3	forward/backward parity, tok/s, memory
dtype	FP32/BF16/FP16/FP8	loss, NaN, error, throughput
sequence length	1K/4K/16K/64K+	latency, memory, effective-context eval
head_dim	64/128/256	kernel eligibility, logit stats
GQA ratio	Hkv=H, H/2, H/4, 1	quality, KV bytes, decode tok/s
mask	causal/window/block	correctness, kernel path
scale/temperature	baseline $\pm$ variants	entropy, loss, stability
QK norm	off/on	norm, logits, convergence

23.1 Benchmark hygiene

- 固定软件版本、GPU 型号、driver/CUDA。
- warmup 后 cuda synchronize 再计时。
- 报告 batch、 L_q 、 L_k 、heads、head_dim、dtype、causal、dropout。
- 把 prefill 和 decode 分开。
- 验证实际 backend，没有 silent fallback。
- 性能提升必须附同精度/同模型输出容差。

24. Debug Playbook：从 NaN 到 kernel mismatch

阶段	动作
----	----

Step 1	切到最小 batch、最短 sequence、FP32 reference。
Step 2	打印/断言 Q,K,V finite；记录 RMS/max。
Step 3	手算 3×3 logits 和 mask，确认允许集合。
Step 4	检查每个 query 至少一个有效 key。
Step 5	比较 pre-softmax logits；再比较 softmax；最后比较 AV。
Step 6	关闭 dropout、关闭 autocast、关闭 fused kernel，逐项恢复。
Step 7	full forward vs cached decode parity。
Step 8	按 layer/head 定位首次误差放大点。
Step 9	若仅性能异常，确认 backend eligibility/fallback/stride/copy。
Step 10	将最小复现固定为 unit test，防止回归。

经验法则

“生成文本看起来正常”不是 attention correctness test。错误 mask、cache offset、head mapping 可能在短 prompt 上偶尔看不出来，却在长上下文/beam/speculative decode 中灾难性放大。

25. 面试与推导题

1. 从独立同分布假设推导为什么 QK^T 要除以 $\sqrt{d_k}$ 。
2. 证明 $\text{softmax}(s+c)=\text{softmax}(s)$ ，并说明为什么这允许 stable softmax。
3. 给出 online softmax 合并两个 block 的 running max/normalizer 公式。
4. 为什么 FlashAttention 是 exact attention，却能把中间内存从 $O(L^2)$ 降到近 $O(L)$ ？
5. 为什么它的算术复杂度仍是 $O(L^2)$ ？
6. 解释 prefill 与 decode 为什么硬件瓶颈不同。
7. 给定 $B=1, H_{kv}=8, D_h=128, L=128K, 32 \text{ layers}, BF16$ ，估算 KV cache。
8. bool mask True 到底代表 allow 还是 deny？你如何写无歧义测试？
9. 为什么全 mask 行会导致 NaN？生产系统如何处理？
10. Attention dropout 后每行是否仍和为 1？为什么？
11. 为什么 attention map 不能直接当 token attribution？
12. 解释 attention sink 对 sliding-window KV eviction 的影响。
13. 为什么 GQA 能加速 decode，却不改变 Q head 数？
14. 什么情况下 kernel 会从 Flash fallback 到 math？为什么必须监控？
15. 如何验证一个自定义 Triton attention kernel 的正确性？
16. 为什么不同 attention kernel 不应要求 bitwise equality？
17. 长上下文下 softmax “稀释”与 RoPE 外推是同一个问题吗？
18. QK-Norm 会如何改变传统 $1/\sqrt{d_k}$ 的解释？
19. FlashAttention backward 为什么允许重算部分中间量？

20. 设计一个 scale / QK-Norm / sequence length 的三因素 ablation。

26. 参考文献与结论

- [1] Vaswani, A. et al. Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.
- [2] Milakov, M.; Gimelshein, N. Online normalizer calculation for softmax. arXiv:1805.02867, 2018.
- [3] Dao, T. et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135, 2022.
- [4] Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691, 2023 / ICLR 2024.
- [5] Shah, J. et al. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. arXiv:2407.08608, 2024.
- [6] Xiao, G. et al. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453, ICLR 2024.
- [7] PyTorch Documentation. torch.nn.functional.scaled_dot_product_attention. Stable docs, accessed 2026-08.
- [8] PyTorch Tutorial. Implementing High-Performance Transformers with Scaled Dot Product Attention. Stable docs, accessed 2026-08.
- [9] Shaw, P.; Uszkoreit, J.; Vaswani, A. Self-Attention with Relative Position Representations. arXiv:1803.02155, 2018.
- [10] Gu, X. et al. When Attention Sink Emerges in Language Models: An Empirical View. arXiv:2410.10781, 2024/2025 revision.
- [11] Ran-Milo, Y. Attention Sinks Are Provably Necessary in Softmax Transformers: Evidence from Trigger-Conditional Tasks. arXiv:2603.11487, 2026. (new theoretical result; scope-specific)
- [12] Velickovic, P. et al. Softmax is not Enough (for Sharp Size Generalisation). arXiv:2410.01104, revised 2025.

最终结论

Scaled Dot-Product Attention 可以同时从四层理解：数学层是“归一化的 content-based weighted read”；优化层是“logit 尺度与 softmax Jacobian 的稳定性”；系统层是“两个大 GEMM + reduction/exp + 极高 HBM traffic”；长上下文层是“位置、候选规模、softmax sharpness 与 KV/IO 成本的共同问题”。真正掌握 Attention，必须把这四层连起来。